

Continuous Batching

目 录

1 引言：静态调度的拖尾效应	2
2 Continuous Batching 的调度循环	2
3 请求生命周期	2
3.1 框架中的 RequestState	3
4 StaticBatchScheduler 的扩展	3
5 Chunked Prefill：长 Prompt 的分块	4
5.1 为什么要分块	4
5.2 Token Budget	4
6 KV Cache 回收	4
7 实战：静态 vs 连续批处理的时间线	5
8 性能影响	6
9 本章你将学会	6
10 要点速查	6
11 小结	7

1 引言：静态调度的拖尾效应

LLM 推理服务的请求通常以异步方式到达，每个请求的 prompt 长度和生成长度各不相同。最简单的调度策略是 **Static Batching**（静态批处理）：从队列中取出最多 B 个请求组成一批，一起处理，等所有请求都完成后才取下一批。

问题在于生成长度的差异。假设一批 4 个请求的生成长度分别为 10、20、5、15 个 token。最短的请求在第 5 步就完成了，但它的 batch 槽位不能被新请求复用，必须空等到第 20 步最长请求结束。随着短请求陆续完成，参与计算的请求数越来越少，GPU 的 batch size 不断下降，但每步的计算开销不变，吞吐量随之衰减。

直觉 想象一条流水线上有 4 个工人同时组装不同的产品。有人 5 分钟做完，有人 20 分钟才做完。做完的人不能离开去帮别人，必须站在工位上干等。工厂付了 4 个人的工资，但后半段时间只有 1 个人在干活。这就是拖尾效应：尾部少数长请求拖住了整个 batch 的吞吐量。

Continuous Batching（连续批处理）的核心思想是：每次迭代后重新调整活跃请求集合，让完成的请求尽早离开、等待的请求尽早加入，使 batch size 长时间保持在高位。

2 Continuous Batching 的调度循环

Continuous Batching 不再等整批完成才换人，而是每生成一个 token 后重新评估当前活跃请求集合。每一轮迭代的调度循环包含四个步骤：

1. **检查完成**：遍历活跃请求，检查本轮生成的 token 是否触发停止条件（EOS 标记或达到最大长度）。将完成的请求移出活跃集合，释放其 KV Cache
2. **接纳新请求**：根据最大 batch size 和剩余 KV Cache 容量，从等待队列中取出新请求加入活跃集合
3. **执行 Prefill**：对新加入的请求执行 prefill，计算 prompt 的 KV Cache 并生成第一个 token
4. **执行 Decode**：对处于 decoding 状态的请求各生成一个 token，进入下一轮调度

这个循环的关键在于：完成一个请求后立刻释放资源，新请求立刻填充进来，batch 槽位不会空等。

```
Iteration 1: [Req1][Req2][Req3][Req4] → decode → 检查
Iteration 2: [Req1][Req2][Req3][Req4] → decode → 检查
Iteration 3: [Req1][Req2][Req3][Req4] → decode → Req3 完成，释放
Iteration 4: [Req1][Req2][Req5][Req4] → Req5 prefill → decode
Iteration 5: [Req1][Req2][Req5][Req4] → decode → Req1 完成，释放
Iteration 6: [Req6][Req2][Req5][Req4] → Req6 prefill → decode
...
```

每一轮迭代后 batch 都保持 4 个活跃请求（假设等待队列非空），GPU 利用率不会因短请求完成而下降。

3 请求生命周期

每个请求在系统中经历四个状态：

状态	名称	含义
PENDING	等待中	请求已入队，尚未被调度器选中
PREFILLING	预填充中	正在执行 prompt 的 prefill 计算
DECODING	解码中	正在逐 token 生成输出
COMPLETED	已完成	触发停止条件，已移出活跃集合

请求从 PENDING 进入 PREFILLING，prefill 完成后进入 DECODING，每次 decode 生成一个 token。当满足停止条件时转入 COMPLETED，释放资源。

3.1 框架中的 RequestState

Lab5 的实验框架用 RequestState 类记录每个请求的状态，核心字段包括：

```
@dataclass
class RequestState:
    prompt_token_ids: list[int]      # 输入 prompt 的 token 序列
    output_token_ids: list[int]      # 已生成的输出 token 序列
    num_computed_tokens: int         # 已完成计算的 token 数
    status: RequestStatus            # PENDING / PREFILLING / DECODING / COMPLETED
    finish_reason: str | None        # 完成原因: "stop" / "length" / None
```

prompt_token_ids 和 output_token_ids 分别记录输入和输出 token。num_computed_tokens 追踪 prefill 进度，用于支持 chunked prefill。status 和 finish_reason 标识请求当前所处阶段和退出原因。

4 StaticBatchScheduler 的扩展

框架提供的 StaticBatchScheduler 是静态调度器：第一次调用以 prefill 模式处理所有 prompt，后续调用以 decode 模式逐 token 生成。它的 update() 方法检查停止条件和最大长度限制，将完成的请求标记为 COMPLETED。

```
class StaticBatchScheduler:
    def schedule(self) -> SchedulePlan:
        if self.first_step:
            return self._schedule_prefill()
        return self._schedule_decode()

    def update(self, outputs: list):
        for i, req in enumerate(self.active):
            token = outputs[i]
            req.output_token_ids.append(token)
            if token == eos_token_id:
                req.status = RequestStatus.COMPLETED
                req.finish_reason = "stop"
            elif len(req.output_token_ids) >= max_tokens:
                req.status = RequestStatus.COMPLETED
                req.finish_reason = "length"
```

逐行说明：schedule 在第一步返回 prefill 计划，之后返回 decode 计划。update 遍历每条输出，将 token 追加到 output_token_ids。若遇到 EOS 标记则设为 COMPLETED，原因为 "stop"；若达到最大长度则同样设为 COMPLETED，原因为 "length"。

实现 Continuous Batching 需要在此基础上扩展：将 schedule 改为可重复调用的调度步，在每步中检查完成、接纳新请求、混合 prefill 和 decode。

5 Chunked Prefill: 长 Prompt 的分块

5.1 为什么要分块

当一个新请求的 prompt 很长时（如 2000 个 token），一次性完成 prefill 的计算量很大，会阻塞正在 decode 的活跃请求，导致它们的延迟突增。**Chunked Prefill**（分块预填充）将长 prompt 切分为多个 chunk，每轮只处理一个 chunk，与 decode 交替执行。

5.2 Token Budget

每轮迭代的计算量由 **Token Budget**（token 预算）限制。例如设定预算为 1024 个 token：如果本轮有 3 个 decode 请求（各 1 个 token，共 3 个），剩余预算 1021 个 token 可以用于一个新请求的 prefill chunk。如果某请求的 prompt 剩余 1500 个 token 未处理，则本轮处理 1021 个，剩余 479 个留到下一轮。

```
Iteration N:
  Decode: Req_A (1 tok) + Req_B (1 tok) + Req_C (1 tok) = 3 tokens
  Prefill chunk: Req_D processes 1021 of 2048 prompt tokens
  Total: 1024 tokens (within budget)

Iteration N+1:
  Decode: Req_A (1 tok) + Req_B (1 tok) + Req_C (1 tok) = 3 tokens
  Prefill chunk: Req_D processes 1021 of remaining 1027 tokens
  Total: 1024 tokens (within budget)
```

分块后，长 prompt 不会独占 GPU，decode 请求的延迟得到保障。代价是该请求的 prefill 跨越多轮完成，TTFT（Time To First Token）略有增加。

Chunked prefill 的 chunk size 选择是一个权衡：chunk 太大则 decode 延迟受影响，chunk 太小则 prefill 效率下降（矩阵规模小，Tensor Core 利用率低）。实际中通常取 128 到 2048 之间。

6 KV Cache 回收

请求完成后，其占用的 KV Cache 必须及时释放。在静态分配方案中，KV Cache 是一块连续的显存区域，释放意味着将该区域标记为可用。在 Paged Attention 方案中，释放意味着将该请求占用的物理块回收到 Block Pool 的空闲列表。

```
def free_request(self, req: RequestState):
    for layer in self.kv_caches:
```

```
layer.free(req.request_id)
req.status = RequestStatus.COMPLETED
```

`free_request` 遍历每一层的 KV Cache，调用 `free` 释放该请求占用的槽位或 block。释放后这些资源立即可供新请求使用。

直觉 KV Cache 回收就像酒店退房：客人离开后房间立刻被打扫并分配给下一位入住的客人。如果不回收，长此以往酒店房间全被离开的客人占着，新客人无房可住，显存 OOM。Continuous Batching 的高效性正是建立在“即走即释放、即来即分配”的快速回收机制上。

7 实战：静态 vs 连续批处理的时间线

例 取 4 个请求，生成长度分别为 10、20、5、15 个 token。最大 batch size 为 4，每步生成 1 个 token。假设等待队列中始终有新请求可接入。

静态调度：4 个请求同时开始 decode，每步生成 4 个 token。第 5 步 Req3 完成（5 个 token），但槽位不能复用。第 10 步 Req1 完成。第 15 步 Req4 完成。第 20 步 Req2 完成，整批结束。

时间线（每格代表一步）：

Step:	1	2	3	4	5	6	7	8	9	10	11-15	16-20
Req1:	D	D	D	D	D	D	D	D	D	D	.	.
Req2:	D	D	D	D	D	D	D	D	D	D	D	D
Req3:	D	D	D	D	D	.	.	.	.	.	.	.
Req4:	D	D	D	D	D	D	D	D	D	D	.	.
Util:	4	4	4	4	4	3	3	3	3	3	2	1

总 wall time = 20 步。实际有效计算 = 10 + 20 + 5 + 15 = 50 token 步。平均 GPU 利用率 = $\frac{50}{4 \times 20} = 62.5\%$ 。

Continuous Batching：第 5 步 Req3 完成后立刻接入 Req5。第 10 步 Req1 完成后接入 Req6。第 15 步 Req4 完成后接入 Req7。每步都保持 4 个活跃请求。

时间线：

Step:	1	2	3	4	5	6	7	8	9	10	11-15	16-20
Req1:	D	D	D	D	D	D	D	D	D	D	.	.
Req2:	D	D	D	D	D	D	D	D	D	D	D	D
Req3:	D	D	D	D	D	.	.	.	.	.	.	.
Req4:	D	D	D	D	D	D	D	D	D	D	.	.
Req5:	.	.	.	.	.	P	D	D	D	D	D	D
Req6:	.	.	.	.	.	.	.	.	.	.	P	D
Req7:	.	.	.	.	.	.	.	.	.	.	.	P
Util:	4	4	4	4	4	4	4	4	4	4	4	4

每步 GPU 利用率始终为 4。同样 20 步内完成 $4 \times 20 = 80$ 个 token 步的有效计算，吞吐量提升 $\frac{80}{50} = 1.6$ 倍。如果队列持续有新请求，GPU 利用率长期维持在 100%。

8 性能影响

Continuous Batching 的核心收益是保持 decode batch size 长期处于高位。由于 decode 阶段每步的计算量与 batch size 成正比但算子发射开销固定，大 batch size 能均摊开销，提高吞吐量。

指标	静态调度	Continuous Batching
平均 batch size	随请求完成递减	始终接近上限
GPU 利用率	$\approx 62\%$	$\approx 100\%$
吞吐量	基准	提升 1.5 到 2 倍
TPOT	受小 batch 影响	大 batch 下更优

Continuous Batching 的 TTFT 可能略高于静态调度，因为新请求的 prefill 与 decode 交替执行，而非独占 GPU。但整体吞吐量的提升远大于 TTFT 的微小增加。

9 本章你将学会

1. 解释静态调度下拖尾效应的成因和 GPU 利用率下降过程
2. 描述 Continuous Batching 调度循环的四个步骤
3. 区分请求的四个生命周期状态及其转换条件
4. 理解框架中 RequestState 的核心字段和 StaticBatchScheduler 的工作方式
5. 将静态调度器扩展为可重复调用的 Continuous Batching 调度步
6. 解释 Chunked Prefill 如何通过 token budget 平衡 prefill 和 decode
7. 实现请求完成后的 KV Cache 回收与资源复用
8. 量化对比静态调度与 Continuous Batching 的 GPU 利用率和吞吐量

10 要点速查

要点	说明
拖尾效应	短请求完成后槽位空等，batch size 递减
Continuous Batching	每步迭代后重新调整活跃请求集合
调度循环	检查完成 $\rightarrow$ 接纳新请求 $\rightarrow$ prefill $\rightarrow$ decode
PENDING	请求已入队，未被调度
PREFILLING	执行 prompt 的 prefill 计算
DECODING	逐 token 生成输出
COMPLETED	触发停止条件，资源已释放
RequestState 字段	prompt/output token ids, num_computed, status
StaticBatchScheduler	首次 prefill，后续 decode，update 检查停止

Chunked Prefill	长 prompt 分块，与 decode 交替执行
Token Budget	每轮总 token 数上限，分配给 prefill 和 decode
KV Cache 回收	请求完成后释放槽位或 block 供新请求复用

11 小结

Continuous Batching 是提升 LLM 推理吞吐量的关键调度策略。它通过每步迭代后重新评估活跃请求集合，让完成的请求尽早释放资源、等待的请求尽早加入计算，使 decode batch size 长期保持在高位。这与静态调度的拖尾效应形成鲜明对比：静态调度下 GPU 利用率随短请求完成而递减，Continuous Batching 下利用率始终接近上限。

在实现层面，调度循环包含检查完成、接纳新请求、执行 prefill 和执行 decode 四个步骤。请求经历 PENDING、PREFILLING、DECODING、COMPLETED 四个状态，框架用 RequestState 记录完整生命周期。Chunked Prefill 通过 token budget 平衡长 prompt 的 prefill 与活跃请求的 decode，避免长 prompt 独占 GPU。KV Cache 的及时回收保证资源不被已完成的请求占据。这些机制组合在一起，使推理引擎能在显存约束下持续保持高吞吐量。

讲义基于 HPC Lab5 实验指导编写